

SEOS — Software Engineering Operating System

An AI-Powered Connected Engineering Workspace

Project Proposal for [Course Code]

Submitted to: [Supervisor Name]

[Institution Name]

[Author Name 1], [Department]

[Author Name 2], [Department]

[Month] [Year]

Roll No: [xxxxxxx], Email: [name]_[batch][institution].edu

1 Higher-order goal ... secondary framing

Software engineering is usually taught as a set of disconnected artifacts — requirements, user stories, UML, database design, architecture, APIs, and documentation — each produced with a different tool and reconciled by hand. The broader aim of this project is to make software engineering education itself more coherent, by showing that these artifacts are relational rather than isolated deliverables. The immediate engineering objective is narrower: build SEOS, an AI-assisted platform that keeps requirements, models, architecture, APIs, and documentation synchronized through a single connected project model, and that explains the reasoning behind the decisions it recommends.

2 Time-to-value ... primary heuristic

- We will deliver a working, demonstrable increment at the end of every month rather than holding integration until the final weeks — a working skeleton (auth, database, CI) in Month 1, followed by idea-to-user-stories in Month 2.
- Fast feedback takes priority over completeness: each module — requirements engine, UML/ER designer, architecture recommender, API designer, documentation engine — is built as a narrow, testable slice before being extended, with CI running on every change.
- Success in the first iteration is defined narrowly: turning an idea into structured, editable, traceable requirements and user stories — before later iterations add modeling, architecture, APIs, and documentation.

3 Problem Statement

Students and early-career developers commonly split software engineering work across disconnected tools: requirements in Word or notes, diagrams in a separate UML tool, ER models in a database tool, APIs specified manually or in Swagger, and documentation written independently in Markdown or Word. Because none of these artifacts share a common source of truth, keeping them consistent requires constant manual synchronization.

- Poor propagation: a change to one requirement does not automatically flow into user stories, diagrams, or API contracts.

- Broken traceability: it is hard to see why a diagram element or endpoint exists, or which requirement it satisfies.
- Stale documentation: SRS/SDD/README files are written once and rarely regenerated as the design evolves.
- Low learning value: each artifact is produced mechanically, without practicing the dependencies between engineering decisions.

Why this matters now (contextual relevance): software engineering courses ask students to produce these artifacts individually, but rarely teach or tool for the dependencies between them, leaving the systems-thinking part of the discipline implicit rather than practiced.

4 Proposed Solution ... Software Proposition

4.1 Overview

Build a web-based AI workspace, SEOS, that takes a one-paragraph software idea and progressively produces a connected set of engineering artifacts:

Idea → Requirements → User Stories → UML/ER Models → Architecture → API Specification → SRS / SDD / README

An AI + retrieval-augmented (RAG) explanation layer operates across every stage, grounding “why this decision” answers in a curated software-engineering knowledge base rather than an unexplained model output.

4.2 Core workflow

- The user enters a natural-language idea; the system asks targeted clarification questions rather than immediately generating a long requirements document.
- Approved requirements are transformed into user stories with acceptance criteria, with full traceability back to source requirements.
- Entities, actors, and interactions are extracted into UML class/sequence diagrams and an ER model on an interactive, editable canvas.
- An architecture (e.g. layered, modular monolith, or microservices) is recommended and explained, based on the project's scale and boundaries.
- REST endpoints and an OpenAPI specification are derived directly from the model.
- SRS, SDD, and README are generated from the shared model and can be regenerated as the model changes (“living documentation”).

4.3 Core innovation — connected engineering model

The central technical idea is that requirements, user stories, diagrams, architecture, APIs, and documentation are not independent AI outputs — they are views over one underlying project model. Diagrams represent model objects; documents and API specs are derived from the same model; and a two-way sync between the diagram editor and the model means edits in either direction propagate correctly. This is what distinguishes SEOS from a collection of unrelated “AI generates X” tools.

5 Solution Approach ... Engineering focus

This is an engineering course project, so the emphasis is on building a reliable connected system and a controlled AI pipeline, rather than on inventing new AI/ML research.

5.1 AI reliability ... non-research

- Each AI capability (requirements, UML, architecture, etc.) uses its own prompt and returns structured JSON validated against a schema before persistence — never a single unconstrained prompt.
- Consistency checks and human approval sit between AI output and the database, so nothing is auto-persisted without review.
- RAG grounds explanations in a curated knowledge base (requirements engineering, UML, architecture, database design, API design, testing) via chunking, embeddings, and PostgreSQL + pgvector similarity search — used to explain established practice, not to produce novel algorithms.

5.2 System architecture

- Frontend: React + TypeScript, Tailwind CSS, an interactive diagram canvas (e.g. React Flow).
- Backend: FastAPI (Python), exposing REST APIs for auth, requirements, user stories, diagrams, architecture, API design, documentation, and the AI/RAG layer.
- Database: PostgreSQL for relational project data, with pgvector for vector search.
- AI orchestration: an LLM API with LangChain, using prompt versioning and evaluation cases.

5.3 CI/CD and fast delivery enabler

GitHub Actions will build and test on every change; Docker will containerize the application; the frontend deploys to Vercel and the backend to Render, so every module can be demonstrated against a real staging environment from Month 1 onward.

6 Evaluation Criterion ... measurable and attributable

6.1 Primary evaluation metric

End-to-end completion rate: the proportion of pilot runs in which a one-paragraph idea is carried through to downloadable SRS/SDD/README without the pipeline breaking.

- Target: automated end-to-end tests pass for $\geq 90\%$ of test ideas, and the pipeline completes for every idea used in the final demonstration.
- Attribution: measured via automated pipeline tests and logged pilot walkthroughs.

6.2 Secondary evaluation metrics

- Model-sync correctness: percentage of diagram/model edits that correctly propagate to downstream artifacts, checked with targeted test cases.
- API validity: percentage of generated OpenAPI specifications that import into Swagger/Postman without manual correction.
- Explanation quality: user-rated relevance and clarity of RAG-grounded explanations (1–5 scale).
- Learning value: self-reported understanding gain from external pilot users after completing a full workflow.

6.3 Pilot validation plan

Recruit at least five external users (ideally students unfamiliar with the project) to run one idea each through the complete pipeline, collect structured feedback on clarity, correctness, and learning value, and log completion and error rates from the pipeline itself.

7 Scalability ... with foresight

- Scalable (theoretically): the system decouples frontend and backend, uses a stateless API design, and applies indexing/pagination on common queries (requirements, diagrams, documents) to support growth in projects and concurrent users.
- Operationally deployable (practically): Docker containerization, a managed PostgreSQL instance, and a GitHub Actions → staging/production pipeline (Vercel + Render) keep deployment repeatable rather than ad hoc.

8 Engine availability heuristic

A mature set of existing tools and services is available, so the team is not inventing infrastructure:

- Web framework: FastAPI for REST APIs.
- Frontend: React + TypeScript + Tailwind, with React Flow for diagram editing.
- Database: PostgreSQL, with pgvector for embeddings and similarity search.
- Authentication: JWT-based tokens.
- AI orchestration: an LLM API with LangChain for prompting and retrieval.
- CI/CD: GitHub Actions; containers via Docker; deployment via Vercel and Render.

Using these mature components lets the team concentrate effort on the connected-model design, synchronization logic, and AI reliability, rather than on infrastructure.

9 Project scope and deliverables ... iteration-friendly

9.1 Initial deliverable (Months 1–2)

- Auth, database schema, base UI, and CI pipeline (Month 1 skeleton demo).
- AI clarification engine: idea → structured requirements → user stories with traceability (Month 2 demo).

9.2 Subsequent deliverables (Months 3–6)

- Interactive UML/ER editor with two-way model synchronization (Month 3).
- Architecture recommender, API designer, RAG integration, OpenAPI validation (Month 4).
- SRS/SDD/README generation and Learning Dashboard, with usability testing (Month 5).
- End-to-end integration, testing, deployment, and final demonstration (Month 6).

9.3 Explicit exclusions (scope control)

To keep the project achievable for a three-person team in six months, the following are deliberately out of scope and treated as future work: an AI code reviewer, automated test-case generation, a full DevOps/CI-CD assistant, a sprint/Kanban planner, and a full AI coding assistant.

10 Team division

- Member 1 — Frontend / UX: workspace UI, diagram editor, Learning Dashboard.
- Member 2 — Backend / Platform: FastAPI services, database, auth, project model, deployment.
- Member 3 — AI / RAG: LLM integration, prompt design, RAG pipeline, knowledge base, AI evaluation.

These are primary ownership areas rather than strict silos; integration work is shared, especially during the final month.

11 Risks and mitigations

- Inconsistent LLM output: mitigated with structured schemas, validation, and prompt test cases.
- Two-way diagram synchronization is the hardest technical component: prototype it early, define a controlled model, and keep a one-way fallback.
- Scope creep: freeze the core pipeline early and classify extras (Section 9.3) as future work.
- Integration bottlenecks across three modules: shared API contracts, weekly integration checkpoints, and a final-month buffer.
- AI hallucination in explanations or artifacts: RAG grounding, validation, test cases, and human approval before persistence.
- User experience complexity: a progressive, staged workflow with in-context explanations and a progress dashboard.

12 Summary

SEOS proposes a deployable, AI-assisted workspace that carries a software idea from a one-paragraph description through requirements, user stories, UML/ER modeling, architecture, API design, and documentation — kept consistent through a single connected project model rather than a set of disconnected AI generators. It meets course criteria by delivering a demonstrable increment every month, using CI/CD for frequent safe releases, and defining measurable, attributable evaluation criteria: end-to-end completion rate, model-sync correctness, API validity, and pilot-tested learning value. Scope is deliberately bounded so that a three-person team can deliver a complete, working system within six months.